

Module 11 - chapter 8

Jenkins Credentials Note on Best Practices

Security concepts and security aspects of DevOps are covered in the [DevSecOps Bootcamp](#) in detail. It's a huge topic on its own. In the bootcamp, you will learn the **security best practices** in DevOps including access management, how to securely deploy from CI/CD with short-lived tokens etc.

In Module 9 chapter 7 and 8

We created an used SSH to access EC2 server from my laptop

```
-> ssh -i ~/.ssh/docker-server.pem ec2-user@18.132.18.175
```

-i -> allows to use a parameter

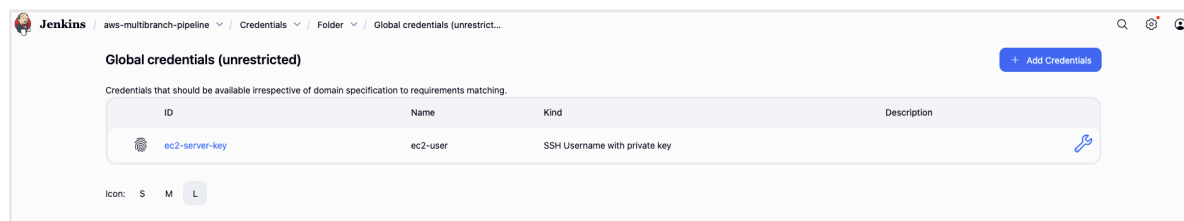
~/.ssh/docker-server.pem -> is the parameter we pass to the command

ec2-user -> there are two users, one is the root user and we don't want to ssh into root we want to ssh into ec2 user and by default in AWS images is ec2-user . If I don't specify the user then by default it thinks is root user.

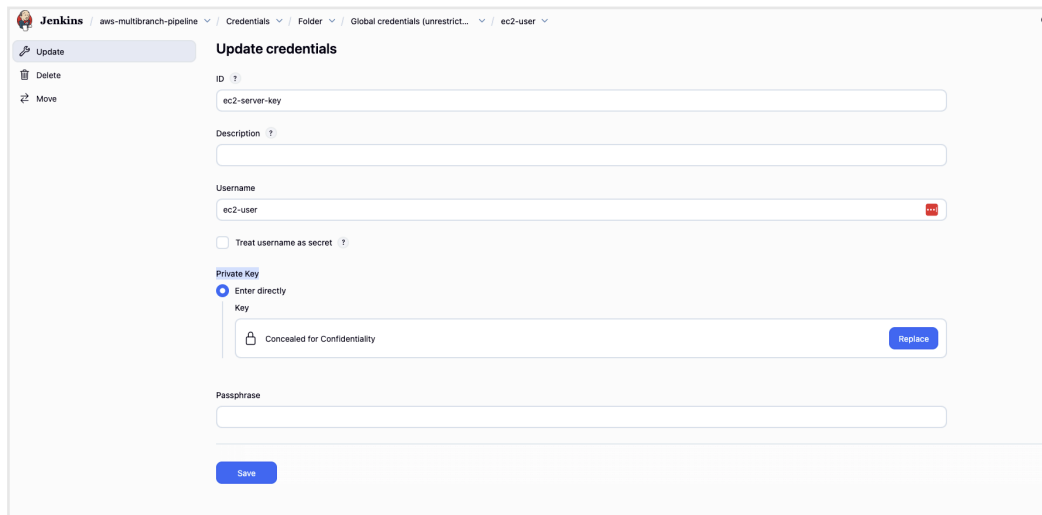
18.132.18.175 -> the public IP address

We accessed the EC2 instance using the EC2 key pair (we created this when creating the EC2 instance) and we ssh into the EC2 instance using the user '**ec2-user**'

and then we did the same for Jenkins via a credential



The credential had the key stored (the key pair was created in AWS by the EC2 instance and downloaded to my laptop) and the user is 'ec2-user':



There is a better alternative:

- Create a Jenkins user on EC2 server
- Create credentials for specifically Jenkins user. We will generate a private key in AWS specifically for Jenkins user and use that private key when creating the credential instead of 'ec2-user'



Better alternative

- ▶ Create Jenkins user on EC2 server
- ▶ Create credentials on Jenkins with its private key

In Module 11 - chapter 7

Nana created credentials in Jenkins with AWS key pairs made by 'admin' user and because of this when the eks cluster is created 'admin' the creator of the cluster gets by default intrinsic access to the cluster and aws-auth file doesn't need edition to add this information to the file. Now Jenkins is using the 'admin' keys and has full access to AWS and full permissions to the cluster



I had to edit was-auth this because Jenkins was not using credentials from 'admin' but from 'jenkins' user and 'jenkins' user didn't create the eks cluster hence it did not have intrinsic permissions to the cluster and then 'jenkins' had to be added to was-auth

```
# Please edit the object below. Lines beginning with a '#' will be ignored,  
# and an empty file will abort the edit. If an error occurs while saving this file will be  
# reopened with the relevant failures.  
#  
apiVersion: v1  
data:  
  mapRoles: |  
    - rolearn: arn:aws:iam::706976333940:role/eksctl-demo-cluster-nodegroup-demo-NodeInstanceRole-vFVE9tMraBd9  
      groups:  
        - system:bootstrappers  
        - system:nodes  
      username: system:node:{{EC2PrivateDNSName}}  
  mapUsers: |  
    - userarn: arn:aws:iam::706976333940:user/jenkins  
      username: jenkins  
      groups:  
        - system:masters  
kind: ConfigMap  
metadata:  
  creationTimestamp: "2026-06-09T08:35:43Z"  
  name: aws-auth  
  namespace: kube-system  
  resourceVersion: "1151"  
  uid: 28c70cb9-0ec1-4349-a0b7-5327573a7069
```

I created a Jenkins user (with restricted AWS permissions) and a key pair specifically for Jenkins user to authenticate but once Jenkins is authenticated it has full permissions for EKS because in the was-auth file I used:

-> system:masters 🙅

— — —

Currently my aws-auth looks like:

```
mapUsers: |
  - userarn: arn:aws:iam::706976333940:user/jenkins
    username: jenkins
    groups:
      - system:masters    ← full admin, too broad
```

Jenkins IAM user authenticated to EKS



aws-auth maps jenkins-ci → system:masters



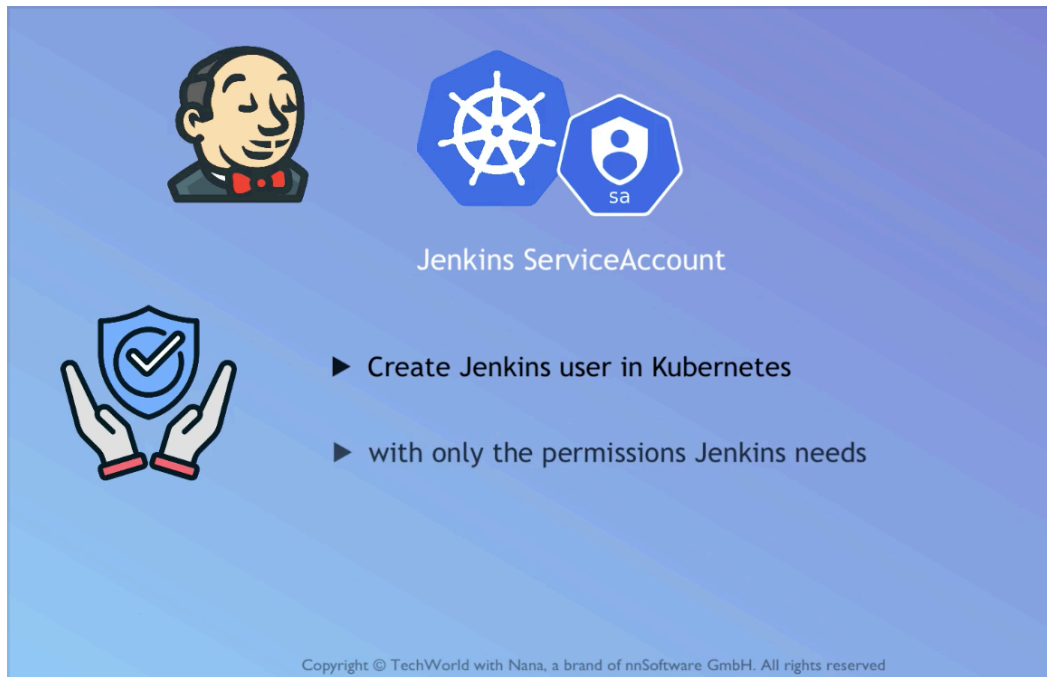
full cluster admin access

It works — Jenkins can create deployments, services, everything.

So it is better to limit the RBAC permissions for Jenkins as well and this is done via a ServiceAccount (if Jenkins is a pod that runs in the EKS cluster):

```
mapUsers: |
  - userarn: arn:aws:iam::706976333940:user/jenkins
    username: jenkins    ← maps IAM user to k8s username "jenkins"
    groups: []           ← no groups, permissions come from RoleBinding instead
```

```
mapUsers: |
  - userarn: arn:aws:iam::706976333940:user/jenkins
    username: jenkins    ← maps IAM user to k8s username "jenkins"
    groups: []           ← no groups, permissions come from RoleBinding instead
```



So we need aws-auth edited and Kubernetes "ServiceAccount" for Jenkins with limited RBAC permissions:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: jenkins-role
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "create", "update", "delete"]
  - apiGroups: [""]
    resources: ["pods", "services"]
    verbs: ["get", "list", "create", "update", "delete"]
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: jenkins-role
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "create", "update", "delete"]
  - apiGroups: [""]
    resources: ["pods", "services"]
    verbs: ["get", "list", "create", "update", "delete"]
```

```
apiVersion: rbac.authorization.k8s.io/v1
```

```

kind: ClusterRoleBinding
metadata:
  name: jenkins-role-binding
subjects:
  - kind: User
    name: jenkins      ← matches username in aws-auth
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: jenkins-role
  apiGroup: rbac.authorization.k8s.io

```

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: jenkins-role-binding
subjects:
  - kind: User
    name: jenkins      ← matches username in aws-auth
    apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: jenkins-role
  apiGroup: rbac.authorization.k8s.io

```

Notice — Subject is User **not ServiceAccount**:

```

subjects:
  - kind: User      ← not ServiceAccount
    name: jenkins   ← the username from aws-auth mapping

```

This is because Jenkins is an external user authenticating via IAM — not a pod running inside the cluster.

ServiceAccount is for pods running inside the cluster.

So technically what your teacher calls "jenkins user for k8s" is really a k8s User bound to a ClusterRole —

not a ServiceAccount in the strict Kubernetes sense.

The two layers working together:

Jenkins pipeline runs

|



Layer 1 — AWS authentication:

aws-iam-authenticator uses IAM user credentials

→ proves to EKS "this is jenkins IAM user"

|



Layer 2 — Kubernetes authorisation:
aws-auth maps jenkins → jenkins-sa
RBAC checks jenkins-sa permissions
→ allowed to create deployments

Layer	Identity	Purpose
AWS	IAM user (jenkins-ci)	authenticate to AWS/EKS
Kubernetes	ServiceAccount (jenkins-sa)	authorise what Jenkins can do in cluster

We need both — the IAM user for AWS authentication and the ServiceAccount for Kubernetes authorisation.

They are not alternatives to each other, they work at different layers.

Where was the yaml to create the Jenkins Service account?

We don't need it because Jenkins is not running as a pod in our cluster.

So creating a Jenkins user in AWS was good practice. Next time I can do better by adding RBAC to Jenkins via a service account - SA.

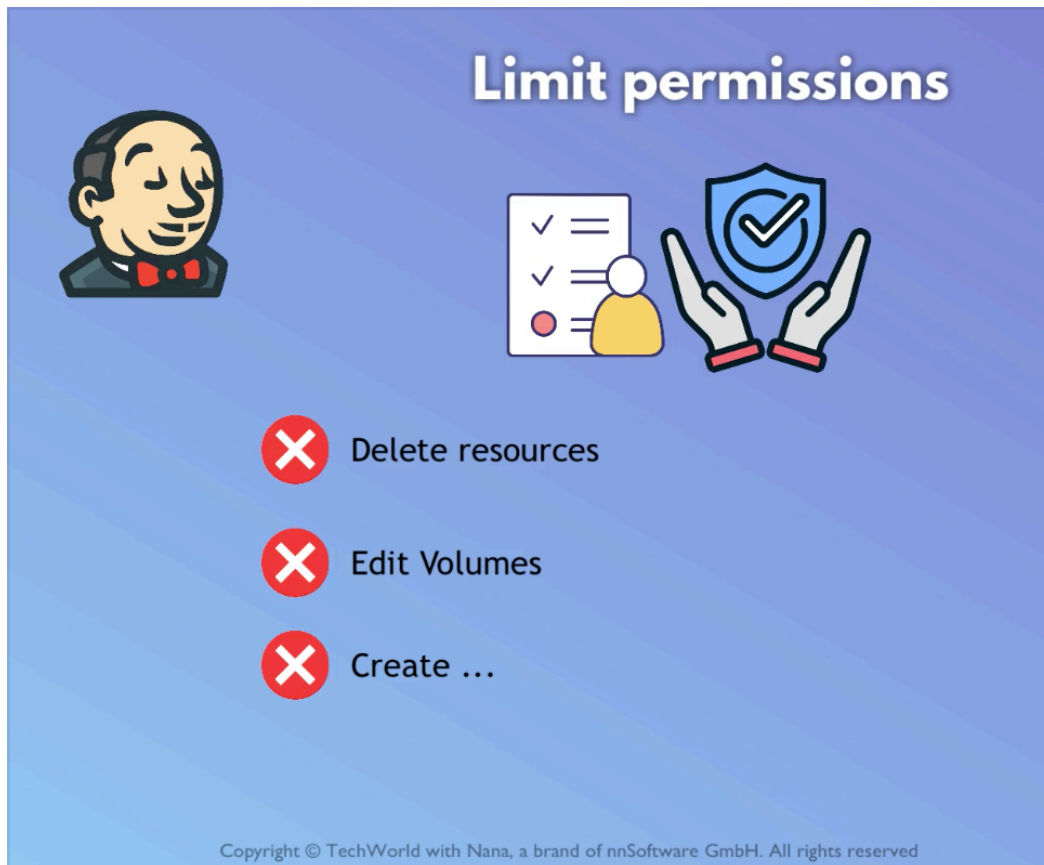


Limit permissions



-  Deploy application to specific namespace
-  Create Deployment and Service resources
-  ...

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved



With a ServiceAccount approach it should be:

If Jenkins runs as a pod in the EKS cluster then we don't need to edit aws-auth file and we create the service account - SA for Jenkins pod:

instead of system:masters (do everything)



jenkins-sa with ClusterRole (do only what pipeline needs)

- create deployments ✓
- update deployments ✓
- create services ✓
- delete cluster ✗
- delete namespaces ✗

And the

apiVersion: v1

kind: ServiceAccount

metadata:

name: jenkins-sa

namespace: default

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins-sa
  namespace: default
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: jenkins-role
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "create", "update", "delete"]
  - apiGroups: [""]
    resources: ["pods", "services"]
    verbs: ["get", "list", "create", "update", "delete"]
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: jenkins-role-binding
subjects:
  - kind: ServiceAccount
    name: jenkins-sa
    namespace: default
roleRef:
  kind: ClusterRole
  name: jenkins-role
  apiGroup: rbac.authorization.k8s.io
```

And the RBAC will point at Jenkins service account 🙌

Jenkins running as a pod inside the cluster:

Jenkins pod starts

|



pod has ServiceAccount: jenkins-sa

|



Kubernetes automatically mounts SA token into pod

|



Jenkins uses SA token to talk to Kubernetes API server directly — no aws-iam-authenticator needed
no ~/.aws/credentials needed

What happens to aws-auth:

aws-auth is not involved at all

Because:

aws-auth bridges: AWS IAM identity → Kubernetes

|

only needed when authenticating FROM OUTSIDE the cluster

|

Jenkins pod is INSIDE the cluster

already has a Kubernetes identity (ServiceAccount)

no need to bridge from AWS IAM

	Jenkins on DigitalOcean (external)	Jenkins pod inside EKS
Authentication	AWS IAM user + aws-iam-authenticator	ServiceAccount token (automatic)
aws-auth needed	yes	no
AWS credentials needed	yes	no (for k8s API calls)
RBAC	ClusterRoleBinding to IAM username	ClusterRoleBinding to ServiceAccount

But one thing still needs AWS credentials **!!**

If Jenkins pod needs to push images to **ECR** or call other AWS services it still needs AWS credentials — but

via IRSA (the OIDC mechanism we discussed) rather than hardcoded keys:

Jenkins pod → Kubernetes API server

uses: ServiceAccount token (no AWS needed)

Jenkins pod → ECR, S3, other AWS services

uses: IRSA (ServiceAccount annotated with IAM role)

So running Jenkins inside EKS is cleaner — no aws-iam-authenticator, no hardcoded AWS credentials for k8s

access, everything handled natively.

User token

Actually we can create a service Account - SA without having a pod **!?**

alternative authentication approach to aws-iam-authenticator. Instead of using AWS IAM credentials, you use a Kubernetes ServiceAccount token directly.

The flow:

```
graph TD
    A[create ServiceAccount in k8s] --> B[generate a long-lived token for that SA]
    B --> C[copy that token]
    C --> D[store it in Jenkins Credentials Store]
    D --> E[Jenkins uses token directly to talk to API server  
no AWS credentials needed  
no aws-iam-authenticator needed]
```

How to create the token:

```
# create the ServiceAccount
kubectl apply -f jenkins-sa.yaml
```

```
# create a long-lived token for it
kubectl create token jenkins-sa --duration=8760h -n default
```

Or create a Secret bound to the SA for a permanent token:

```
apiVersion: v1
kind: Secret
metadata:
  name: jenkins-sa-token
  namespace: default
  annotations:
    kubernetes.io/service-account.name: jenkins-sa
type: kubernetes.io/service-account-token
```

```
# get the token value
kubectl get secret jenkins-sa-token -o jsonpath='{.data.token}' | base64 --decode
```

Store in Jenkins Credentials Store:

Jenkins → Manage Jenkins → Credentials

Kind: Secret text

ID: jenkins-k8s-token

Secret: <paste the token>

Update kubeconfig to use the token:

users:

- name: jenkins-k8s-user

user:

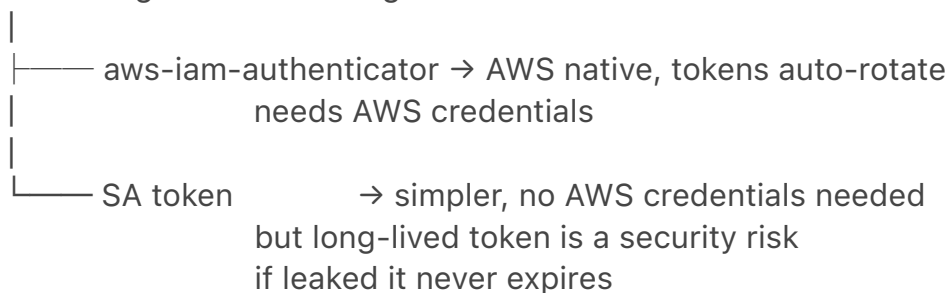
token: <the SA token> ← instead of aws-iam-authenticator

The two approaches compared:

	aws-iam-authenticator	SA token
Needs AWS credentials	yes	no
Token expires	yes (1 hour, auto-refreshed)	no (long-lived)
Works outside AWS	no	yes
tied to AWS IAM	yes	no
aws-auth needed	yes	no

Which approach is better:

Jenkins on DigitalOcean talking to EKS:



So to directly answer your question:

This approach is for external Jenkins — it is an alternative to aws-iam-authenticator. Instead of

authenticating through AWS IAM, Jenkins authenticates directly to the Kubernetes API server using the SA

token.

For Jenkins running as a pod inside the cluster this is not needed — the SA token gets mounted automatically.

Create Jenkins user for K8s

Kubernetes User with a token.

The flow is:

create a k8s User (via certificate or token)



bind it to a ClusterRole via ClusterRoleBinding



generate a token for that user



store token in Jenkins Credentials Store



Jenkins uses token to authenticate to API server directly



Different ways of doing things

NOTE !!

However in practice:

Many people loosely use ServiceAccount tokens for external tools like Jenkins even though technically SAs are meant for pods. It works because:

- SA token is just a JWT
- API server accepts it
- RBAC applies to it

Kubernetes doesn't enforce that SA tokens can only be used by pods — it's a convention not a hard rule.

So the honest picture:

Approach	Technically correct	Works in practice
SA token for external Jenkins	not ideal	yes
k8s User with certificate	correct	yes
aws-iam-authenticator	correct for EKS	yes

Nana is likely using SA token for simplicity — it's a common shortcut even if not strictly correct by Kubernetes conventions.

Best practices:



Questions

Q:

I gave Jenkins user these permissions:

- AmazonEKSClusterPolicy
- AmazonEC2ContainerRegistryFullAccess

A:

- Those are reasonable for learning but let me explain what each gives:

AmazonEKSClusterPolicy:

allows:

- eks:DescribeCluster
- eks:ListClusters
- ec2:DescribeInstances
- ec2:DescribeSecurityGroups
- ec2:DescribeSubnets
- ec2:DescribeVpcs
- and more EC2 read permissions

This is actually designed for the EKS control plane role — not for human/CI users. It gives more permissions than Jenkins strictly needs. For Jenkins you really only need eks:DescribeCluster to authenticate.

AmazonEC2ContainerRegistryFullAccess:

allows:

- ecr:* (everything on ECR)
- create/delete repositories
- push/pull images
- manage lifecycle policies

Too broad for production — Jenkins only needs to push/pull images, not create or delete repositories. But fine for learning.

For **production the least privilege** approach would be:

custom policy with only:

- eks:DescribeCluster
- ecr:GetAuthorizationToken
- ecr:BatchCheckLayerAvailability
- ecr:PutImage
- ecr:InitiateLayerUpload
- ecr:UploadLayerPart
- ecr:CompleteLayerUpload
- ecr:BatchGetImage

For your learning setup — what you have is fine. The important thing is you understand the principle: give only what's needed, nothing more. You can tighten the permissions later when you move to production.

Q:

if I had not created the jenkins aws user and used my aws 'admin' user would I have needed to add this info the aws-auth anyway?

A:

No — because you (the cluster creator) already have implicit access.

When you ran `eksctl create cluster`:

your admin IAM user created the cluster



EKS automatically gives the cluster creator
full admin access



no aws-auth entry needed

This is the exception we discussed earlier — the cluster creator gets implicit
system:masters access without
being in aws-auth.

So if you had used your admin IAM user for Jenkins:

Jenkins uses your admin AWS credentials



aws-iam-authenticator authenticates to AWS as your admin user



EKS recognises: this is the cluster creator



full access granted automatically
no aws-auth entry needed

It would have just worked from the start — which is exactly why using your
personal admin credentials for
Jenkins is tempting but bad practice:

	Personal admin user	Dedicated jenkins-ci user
aws-auth entry needed	no (cluster creator)	yes
Access level	full admin (implicit)	only what you grant
If credentials leak	entire AWS account at risk	limited blast radius
Audit trail	mixed with your activity	separate jenkins activity

So no aws-auth needed for your admin user — but that's precisely why you
shouldn't use it for Jenkins.